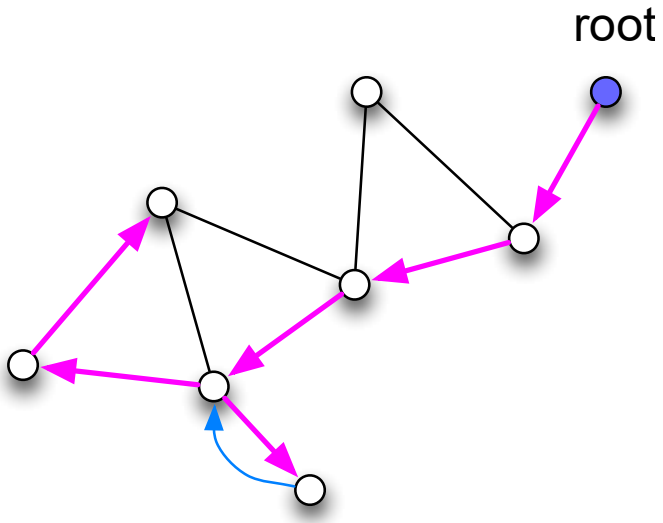
Lecture 1: Introduction & Mathemetical Logic

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

What is Computer Science?

- Computer needs algorithms
  - You may know many basic algorithms



- What is computational theory?



What is Computer Science?

- Computer needs algorithms
  - You may know many basic algorithms

- Any problem impossible to solve by a computer?
- Any problem impossible to solve efficiently by a computer?
- What is a computer/algorithm?



What is Computer Science?

- Any problem impossible to solve by a computer?
  - Yes
  - Computational theory, the first part of this course
- Any problem impossible to solve efficiently by a computer?
  - We are not sure, but we can classify the problems
  - Complexity theory, the second part of this course
- What is a computer/algorithm?



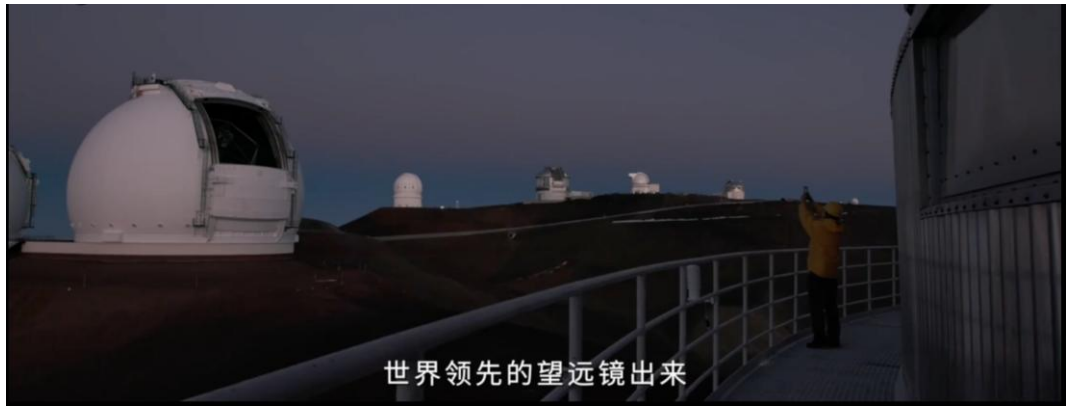
Prerequisite

- 2nd year Yao course
  - After the courses of "Algorithm Design"
  - So you'd better have some knowledge on Algorithms and NP-Completeness



Computer Science is no more about computers than astronomy is about telescopes

-- attributed to Edsger Dijkstra





Textbook

- Introduction to the Theory of Computation
  - Michael Sipser, second or third edition



- Computational Complexity, A Modern Approach
  - Sanjeev Arora and Boaz Barak
- Mathematical Logic part:
  - Logic for Mathematicians, A.G.Hamilton
- Some materials are not in textbooks
- References:
  - See learn.tsinghua.edu.cn



Grading

- 4~5 problem sets
  - Counts for 30% of the final grade
- Final exam
  - Counts for 70% of the final grade
- Bonus points
  - Every problem set has a bonus problem



Today's Lecture

- Overview
- Mathematical Logic



What kind of functions are computable?

- Example:
  - f(x,y)=x+y: given integers x and y ✅
  - f(n): given an positive integer n, f(n)=1 if $x^n + y^n = z^n$ has an positive integer solution, otherwise f(n)=0 ❓ ✅

Also computable: f(n)=1 for n=1,2; otherwise f(n)=0 from Fermat's last theorem, but people use 358 years to prove that

- Whether a function is computable is not obvious



What kind of functions are computable?

- Example of computable functions:
  - f(x,y)=x+y: given integers x and y ✅
  - f(n): given an positive integer n, f(n)=1 if $x^n + y^n = z^n$ has an positive integer solution, otherwise f(n)=0 ✅
- Example of uncomputable functions:
  - Given an integer-coefficient polynomial, determine whether it has integer solutions

$$x^3 + y^3 + z^3 = 42$$



Beginning of Computer Science

- **Hilbert's 10th problem:**
  - Find an algorithm to determine whether a given polynomial Diophantine equation with integer coefficients has an integer solution
  - Two or more variables

Hilbert's problems are a list of twenty-three open problems in mathematics published by German mathematician David Hilbert in 1900





- You can try all the possibilities:
  - x=0,1,-1,2,-2,3,-3,…
  - y=0,1,-1,2,-2,3,-3,…
  - z=0,1,-1,2,-2,3,-3,…

例：

$$x^3+y^3+z^3=42$$

存在：$(-80538738812075974)^3 + 80435758145817515^3 + 12602123297335631^3=42$

（2019年发现）



- You can try all the possibilities:
  - x=0,1,-1,2,-2,3,-3,…
  - y=0,1,-1,2,-2,3,-3,…
  - z=0,1,-1,2,-2,3,-3,…

$$x^3 + y^3 + z^3 = 42$$

- But is that an algorithm?
  - An algorithm has to terminate in finite steps
  - But this procedure cannot terminate if there are no solutions
- Surprisingly, people haven't found any algorithm for this problem, but have proved that such an algorithm does not exist *[Matiyasevich 1970]*



What kind of input and output can be?

- We only consider the functions whose inputs and outputs are all natural numbers **N** = {0, 1, 2, 3, ...}
  - WHY?

  - For Diophantine equation, the input is a polynomial
  - In fact, the input and output can be any type of information



Strings

- An ***alphabet***, Σ, is a finite set of symbols, e.g.,
  - $\Sigma = \{a,\ldots,z\}$
  - $\Sigma = \{0,\ldots,9,+,-,\times,/,(,)\}$
- A ***string*** is a sequence of zero or more symbols from Σ
  - The empty (zero length) string is called "ε"



- Why natural numbers and strings are equivalent?

- Efficient encoding:
  - Σ = {a,b,c}
  - *(lexicographical order)*

0	1	2	3	4	5	6	7	8	9	10	11	12	13	
ε	a	b	c	aa	ab	ac	ba	bb	bc	ca	cb	cc	aaa	…



Coding

- In modern computers, everything is encoded by strings, and stored as natural numbers.







Cardinality of sets

- $|\{a,b,c\}| = 3 \quad \neq \quad 4 = |\{5,7,9,11\}|$

- $|\mathbb{N}|=\infty, |\mathbb{Q}|=\infty, |\mathbb{R}|=\infty,$ so $|\mathbb{N}| = |\mathbb{Q}| = |\mathbb{R}|$??
  - $|\mathbb{N}|$ = natural numbers {0,1,2,3,…}
  - $|\mathbb{Z}|$ = integers {…,-2,-1,0,1,2,…}
  - $|\mathbb{Q}|$ = rationals {2/3, 4, 99/100, 0, 1/33, …}
  - $|\mathbb{R}|$ = reals



Cardinality of sets

- Two sets A, B has the same cardinality if there is a bijective function f: A→B

"injective"：单映射
"surjective"：满映射
"bijective"：一一映射

- The set of integers and the set of even integers have the same cardinality
  - (by f(n)=2n)



Countable set

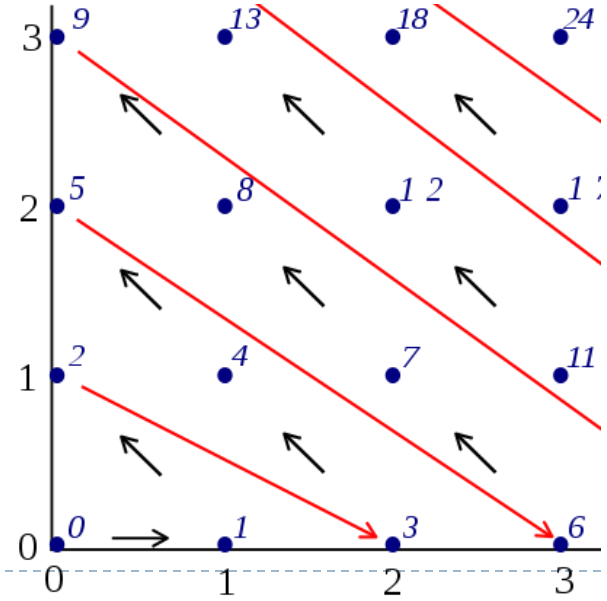
- Countable set: a set S is called **countable** if there exists an injective function f from S to the natural numbers $\mathbb{N} = \{0, 1, 2, 3, ...\}$.
  - Or "if you can list all members of S"

- Easy to see:
  - Set of even numbers is countable
  - Set of integers is countable
  - Set of strings in any alphabet is countable



Countable Sets

- The integers ($\mathbb{Z}$) are countable
  - ***N***: 0 1 2 3 4 5 6 7 …
  - ***Z***: 0 1 -1 2 -2 3 -3 4 …
- Are rational numbers ($\mathbb{Q}$) countable?
  - Yes





We have known:

- The rational numbers ($\mathbb{Q}$) are countable
- Any k-tuple of integers are countable
- Strings in any alphabet are countable
- What about all finite sequence of integers?



We have known:

- The rational numbers ($\mathbb{Q}$) are countable
  - a pair of integers
- Any k-tuple of integers are countable
  - Length is bounded by k, every number is not bounded
- Strings in any alphabet are countable
  - Length is not bounded, alphabet is bounded

- What about all finite sequence of integers?
  - Length is finite but not bounded



Gödel numbering

- Encode a sequence of natural numbers $x_1, x_2, \ldots, x_n$ to one natural number
- Original idea:
  - p_i: the i-th prime, $\{p_i\}=\{2, 3, 5, 7, 11, 13, \ldots\}$
  - x_i=the exponent of p_i of the code number
  - (if there are 0s) the code number $a = {p_1}^{x_1+1} {p_2}^{x_2+1} \ldots {p_n}^{x_n+1}$

- This is a bijective function from a sequence of natural numbers to a natural number
  - Fundamental theorem of arithmetic: unique factorization

Of course, today's computer has many more efficient ways to encode information



What about real numbers

- A real is represented by an infinite seq. of digits
  - (1) Assume there is a complete list of reals in the range [0,1)

r_1:	0.	1	2	1	5	6	6
r_2:	0.	2	3	3	9	9	7
r_3:	0.	4	5	6	7	1	1
r_4:	0.	3	2	8	6	4	5
r_5:	0.	3	4	1	7	7	5
r_6:	0.	4	2	4	3	2	3
⋮							⋱

...



The Reals are Uncountable

- A real is represented by an infinite seq. of digits
- Proof by contradiction: ***(Diagonalization)***
  - (1) Assume there is a complete list of reals in the range [0, 1)

r_1:	0.	1	2	1	5	6	6	...
r_2:	0.	2	3	3	9	9	7	
r_3:	0.	4	5	6	7	1	1	
r_4:	0.	3	2	8	6	4	5	
r_5:	0.	3	4	1	7	7	5	
r_6:	0.	4	2	4	3	2	3	
⋮								⋱

r_1:	0.	1	2	1	5	6	6	
r_2:	0.	2	3	3	9	9	7	
r_3:	0.	4	5	6	7	1	1	...
r_4:	0.	3	2	8	6	4	5	
r_5:	0.	3	4	1	7	7	5	
r_6:	0.	4	2	4	3	2	3	
⋮								⋱
r^*:	0.	2	4	7	7	8	4	?

  - (2) Find a real that is not in the list, a contradiction.
  - Let r* be any real that differs from r_k in the k-th digit. (not 0 or 9)
  - Can r* be in the list?



- Strings (of finite lengths) are countable:
  - Sentences, arguments, proofs in any languages

- Thus, although real numbers are uncountable, only countable number of them can be comprehended by human, like:

$$\sqrt{2},\ \arcsin\left(\frac{2}{23}\right),\ \text{or}\ \int_0^1 x^x\ dx$$

  - "There is a string to describe it"



Computable functions

- How to define computable functions f: $\mathbb{N} \rightarrow \mathbb{N}$
- In 1933, Austrian-American mathematician Kurt Gödel, with Jacques Herbrand, created a formal definition of a class called (general) recursive functions.
  - Try to define "computable functions"



Primitive recursive functions *[Gödel 1931]*

- Functions on natural numbers $\mathbb{N} \rightarrow \mathbb{N}$ generated by:
- Basic functions:
  - zero function: $f(n)=0 \quad \forall n$
  - successor function: $f(n)=n+1 \ \forall n$
  - projection functions: p^k_i: $p^k_i(n_1,\ldots,n_k)=n_i$,
- Two rules:
  - Composition. Given the functions $g(n_1,\ldots,n_j)$ and $h_i(n_1,\ldots,n_k)$ for $1 \leq i \leq j$, then we have

    $f(n_1,\ldots,n_k)=g(h_1(n_1,\ldots,n_k),\ldots, h_j(n_1,\ldots,n_k))$.
  - Recursion. Given $g(n_1,\ldots,n_k)$ and $h(n_1,\ldots,n_{k+2})$, we have

    $f(n_1,\ldots,n_k,0)= g(n_1,\ldots,n_k)$

    $f(n_1,\ldots,n_k,n+1)= h(n_1,\ldots,n_k,n, f(n_1,\ldots,n_k,n))$



Example

- Show that the sum function $f(m,n)=m+n$ is primitive recursive
  - $f_1(n)=n+1$ (successor function)
  - $f_2(n)=n$ (one-variable projection function)
  - $f_3(m,n,p)=p$ (projection function)
  - $f_4(m,n,p)=f_1(f_3(m,n,p))=p+1$ (composition rule)
  - $f_5(m,0)=f_2(m)$

    $f_5(m,n+1)=f_4(m,n,f_5(m,n))$ (recursion rule)

- Show that the minimum function $f(m_1,\ldots,m_n)=\min\{m_1,\ldots,m_n\}$ is primitive recursive
  - not easy



- Does primitive recursive functions include all computable functions?

Basic functions:

zero function: $f(n)=0 \ \forall n$

successor function: $f(n)=n+1 \ \forall n$

projection functions: p^k_i: $p^k_i(n_1,\ldots,n_k)=n_i$,

Two rules:

Composition. Given the functions $g(n_1,\ldots,n_j)$ and $h_i(n_1,\ldots,n_k)$ for $1\le i\le j$, then we have

$f(n_1,\ldots,n_k)=g(h_1(n_1,\ldots,n_k),\ldots, h_j(n_1,\ldots,n_k))$.

Recursion. Given $g(n_1,\ldots,n_k)$ and $h(n_1,\ldots,n_{k+2})$, we have

$f(n_1,\ldots,n_k,0)= g(n_1,\ldots,n_k)$

$f(n_1,\ldots,n_k,n+1)= h(n_1,\ldots,n_k,n, f(n_1,\ldots,n_k,n))$



Ackermann function

- Ackermann function:

$$A(m,n) = \begin{cases} n+1 & \text{if } m = 0 \\ A(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m-1, A(m,n-1)) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

- introduced by Wilhelm Ackermann in 1928.
  - a computable function but not primitive recursive



$A(0, n) = n + 1$

$A(1, n) = 2 + (n + 3) - 3$

$A(2, n) = 2 \times (n + 3) - 3$

$A(3, n) = 2^{n+3} - 3$

$A(4, n) = \underbrace{2^{2^{2^{\cdots 2}}}}_{(n+3 \text{ terms})} - 3$

$\cdots$

Terms of the form $2^{2^{2^{\cdots 2}}}$ are known as power towers.



Basic functions:

zero function: $f(n)=0 \ \forall n$

successor function: $f(n)=n+1 \ \forall n$

projection functions: p^k_i: $p^k_i(n_1,\ldots,n_k)=n_i$,

Two rules:

Composition. Given the functions $g(n_1,\ldots,n_j)$ and $h_i(n_1,\ldots,n_k)$ for $1\le i\le j$, then we have

$f(n_1,\ldots,n_k)=g(h_1(n_1,\ldots,n_k),\ldots, h_j(n_1,\ldots,n_k))$.

Recursion. Given $g(n_1,\ldots,n_k)$ and $h(n_1,\ldots,n_{k+2})$, we have

$f(n_1,\ldots,n_k,0)= g(n_1,\ldots,n_k)$

$f(n_1,\ldots,n_k,n+1)= h(n_1,\ldots,n_k,n, f(n_1,\ldots,n_k,n))$

- Primitive recursion functions are generated by finite number of recursions
- But the Ackermann function A(m,n) needs *unbounded* m number of recursions



- Does primitive recursive functions include all computable functions?

Basic functions:

zero function: $f(n)=0 \ \forall n$

successor function: $f(n)=n+1 \ \forall n$

projection functions: p^k_i: $p^k_i(n_1,\ldots,n_k)=n_i$,

Two rules:

Composition. Given the functions $g(n_1,\ldots,n_j)$ and $h_i(n_1,\ldots,n_k)$ for $1\le i\le j$, then we have

$f(n_1,\ldots,n_k)=g(h_1(n_1,\ldots,n_k),\ldots, h_j(n_1,\ldots,n_k))$.

Recursion. Given $g(n_1,\ldots,n_k)$ and $h(n_1,\ldots,n_{k+2})$, we have

$f(n_1,\ldots,n_k,0)= g(n_1,\ldots,n_k)$

$f(n_1,\ldots,n_k,n+1)= h(n_1,\ldots,n_k,n, f(n_1,\ldots,n_k,n))$

- Key observation: the primitive recursive functions are always well-defined on all natural numbers
  - that's called "total functions"
  - If the function may be not defined on some inputs, it is called "partial functions"



Computable functions

- Since the derivation of a primitive recursive function is a finite string of symbols, we can easily list all primitive recursive functions

- Let f_n is the n-th primitive recursive function in the list
  - we can get this function efficiently from n
  - what about $g(n)=f_n(n)+1$

- Then $g(n)=f_n(n)+1$ is definitely computable, but not in the list

Thus, we need other ideas to define computability



Partial recursive functions

- Functions on natural numbers $\mathbb{N} \to \mathbb{N}$
- Basic functions
- Three rules:
  - Composition.
  - Recursion.
  - Minimisation. Given $g(m,n_1,\ldots,n_k)$, we define

    $f(n_1,\ldots,n_k)=p$ iff $g(p,n_1,\ldots,n_k)=0$, and $g(i,n_1,\ldots,n_k)>0$ for $0 \leq i < p$



Partial recursive functions

- Minimisation. Given $g(m,n_1,\ldots,n_k)$, we define

  $f(n_1,\ldots,n_k)=p$ iff $g(p,n_1,\ldots,n_k)=0$, and $g(i,n_1,\ldots,n_k)>0$ for $0\leq i<p$

- It's like codes:

```
p=0;
while (g(p,n1,…,nk)>0)
        p++;
```

- you cannot guarantee it will "halt", so for function g and inputs $n_1,\ldots,n_k$, f may be not well defined



Recursive functions

- A recursive function is a partial recursive function which is total
  - total: defines on all natural numbers
  - equivalent to there is a Turing machine which computes it
  - the Turing machine halt on all inputs



- Try to construct a counter example by diagonalization as before:

  - We list all recursive functions: (*partial recursive functions which are total*) $f_1, f_2, f_3, \ldots$
  - Find a computable function not in the list: $g(n)=f_n(n)+1$

  - Is it correct?



- Try to construct a counter example by diagonalization as before:

  - We list all recursive functions: (*partial recursive functions which are total*) $f_1, f_2, f_3, \ldots$
  - Find a computable function not in the list: $g(n)=f_n(n)+1$

- Is it correct?

  - You can "list" all partial recursive functions, but you don't know which are total recursive functions
  - "Halting problem" for Turing machine



Recursive sets and relations

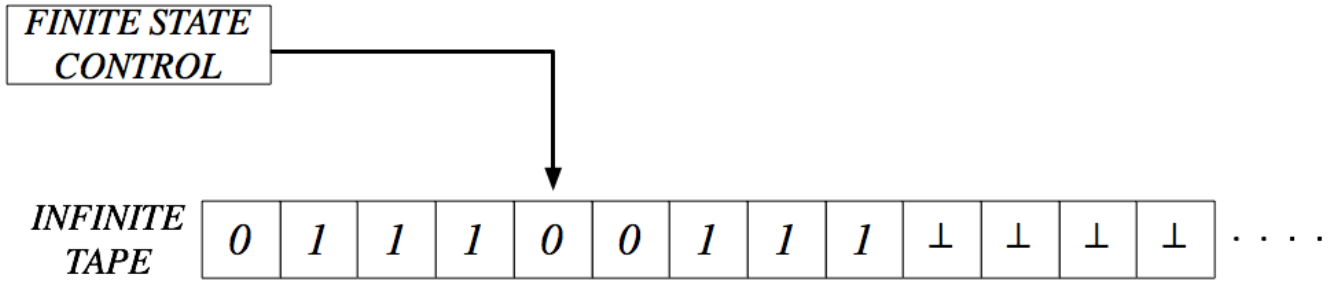
- a set $A \subseteq \mathbf{N}$ is recursive iff its *characteristic function* C_A is recursive:
  - $C_A(x) = 1$ if $x \in A$
  - $C_A(x) = 0$ if $x \notin A$

- a relation R is a subset of $\mathbf{N}^n$ for $n \in \mathbf{N}$
  - e.g. equality "=": {(0,0), (1,1), (2,2), (3,3), …}
  - so we can also define recursive relations. (*there is a recursive function which can check if the relation is satisfied*)



Turing Machines

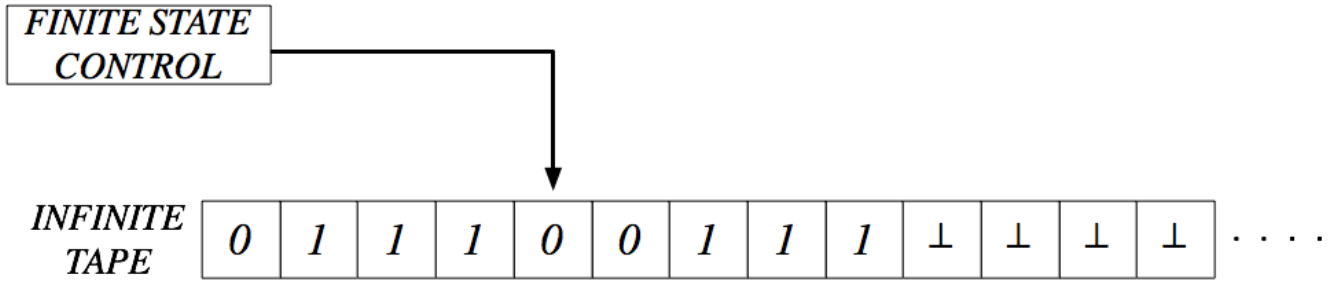
- Give a finite automaton an infinite memory:
  - The memory is an ***infinite tape*** divided into cells
  - One symbol written in each cell (or left blank: ⊥)
  - The automaton has a ***read/write*** "head" at some location on the tape
  - The head can move ***left or right***





Turing Machines

- ***In the beginning…***
  - The input ***w*** is written on the first ***|w|*** cells
  - All other cells are ⊥
  - The machine is in a special ***start state***
  - The head is at the ***leftmost cell***



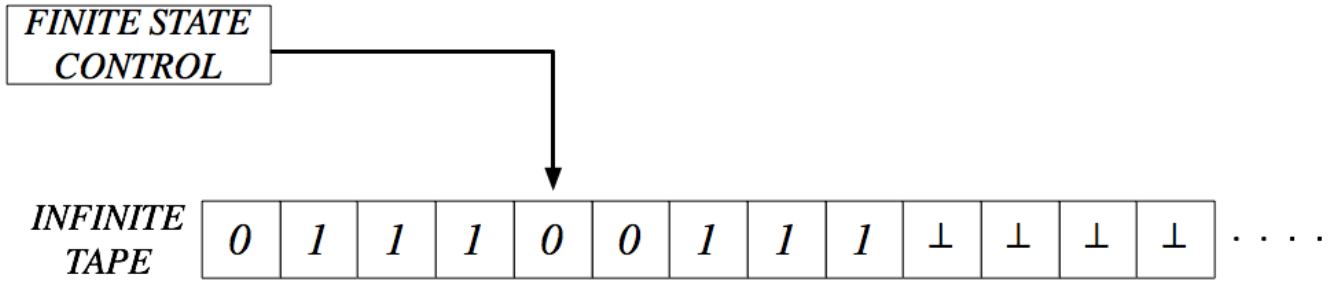


Turing Machines

- At each time step the automaton:
  (1) is in some state
  (2) reads the symbol under the head
  (3) writes a symbol *(possibly the same one)* under the head
  (4) moves left or right
  (5) transitions to a new state *(possibly the same state)*

these depend only on (1,2)

- Two special states: ***accept*** and ***reject***
  - When either of these are entered the machine halts





Three outcomes

- For today you can think of Turing machine as any type of computer

- ***NOTE:*** 3 different outcomes
  - The TM accepts (the input is in the TM's language)
  - The TM rejects
  - The TM runs forever

- A T.M. ***M*** recognizes a language ***A*** if
  - The language of ***M*** is ***A***
- A T.M. ***M*** decides a language ***A*** if
  - ***M*** accepts all w $\in$ ***A*** *and* ***M*** rejects all w $\notin$ ***A*** (i.e., it always halts)



Definitions

- Defn. M ***recognizes*** a language A if L(M) = A
- Defn. M ***decides*** A if L(M)=A and M *always* halts

- A language is ***recursively enumerable*** if some Turing machine recognizes it
- A language is ***recursive*** if some Turing machine decides it

- A TM that always halts describe an algorithm



Halting Problem

- Given a Turing machine M and its input w, decide whether M halts on w
  - Example:

```
n ← 2
Repeat
        n ← n+2
Until n is not the sum of two prime numbers
```

Still unknown (Goldbach's conjecture 1742)

- Turing proved halting problem is not recursive
  - No Turing machine can answer the result for all inputs in finite steps



Halting Problem

- Given a Turing machine M and its input w, decide whether M halts on w
- Turing proved halting problem is not recursive
  - No Turing machine can answer the result for all inputs in finite steps
- But this problem is recursively enumerable
  - so there is a language which is recursively enumerable but not recursive



Back to Turing's paper

- *"On Computable Numbers with an application to the Entscheidungsproblem"*

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbrous technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of



Turing's paper

- "On Computable Numbers, with an Application to the *Entscheidungsproblem*" (submitted on 28 May 1936 and delivered 12 November)
  - Halting problem is undecidable
  - Reduce halting problem to Entscheidungsproblem ("Decision problem")

- In fact, in 1936, Alonzo Church and Alan Turing published independent papers showing that a general solution to the Entscheidungsproblem is impossible
  - TM: simple model to make the reduction simple



Computable functions

- How to define computable functions f: **N** → **N**
- In 1933, Austrian-American mathematician Kurt Gödel, with Jacques Herbrand, created a formal definition of a class called (general) recursive functions.
- In 1936, Alonzo Church created a method for defining functions called the λ-calculus.
- Also in 1936, before learning of Church's work, Alan Turing created a theoretical model for machines, now called Turing machines.

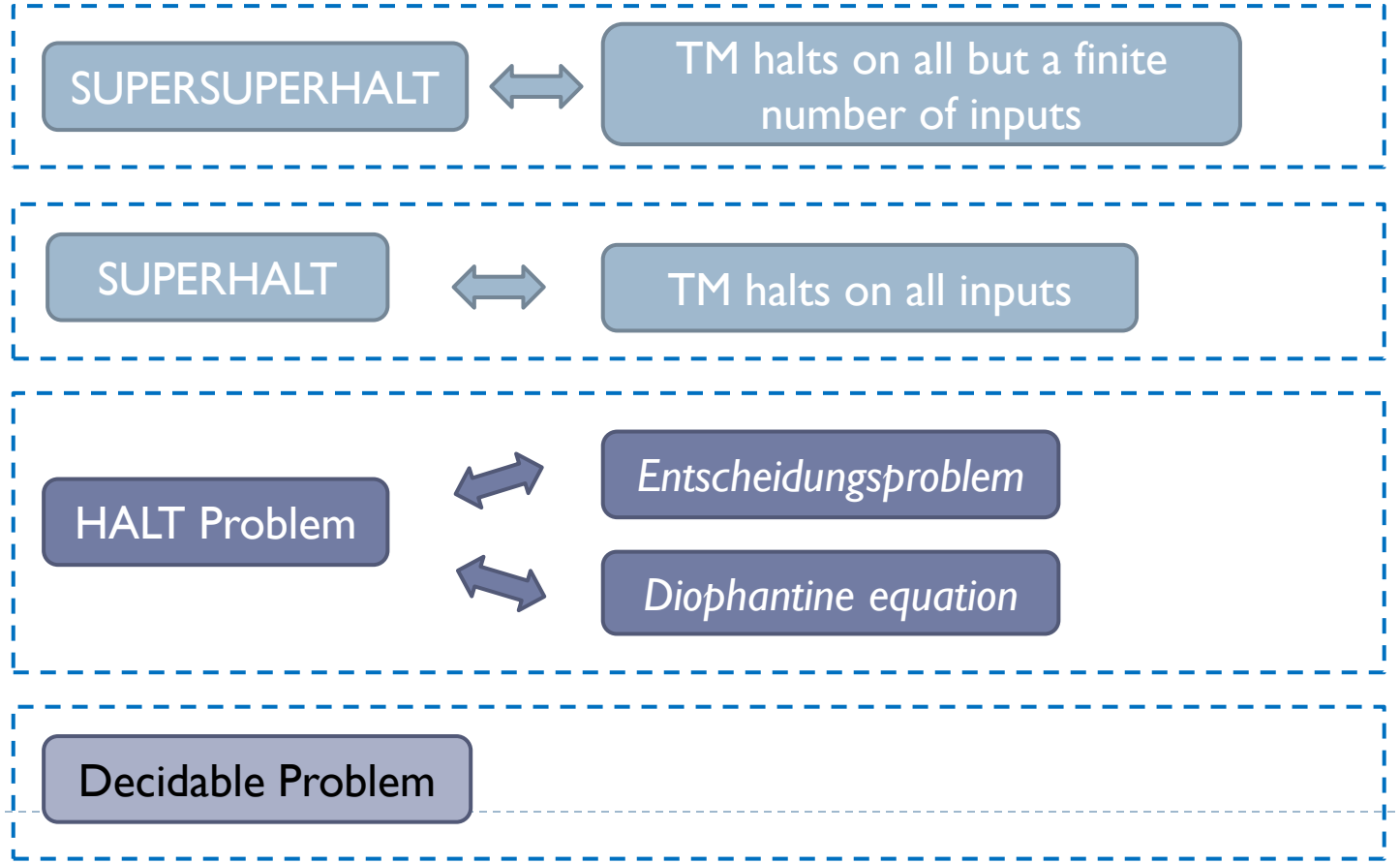


- Church and Turing proved that these three formally defined classes of computable functions coincide:
  - a function is λ-computable if and only if it is Turing computable if and only if it is *(general) recursive*.
  - This has led mathematicians and computer scientists to believe that the concept of computability is accurately characterized by these three equivalent processes.
  - (Church-Turing thesis)
- However, Turing machine is more comprehensible and closer to real algorithms, so usually "recursive" functions are defined from Turing machines.



Problem harder than halting problem?

- SUPERHALT is the halting problem for TMs if we can solve halting problem
- SUPERSUPERHALT…



Complexity class

- Class P: can be solved in polynomial time
  - Eulerian path, shortest path, sorting, max flow, etc.
  - $O(n^c)$ where c is a constant

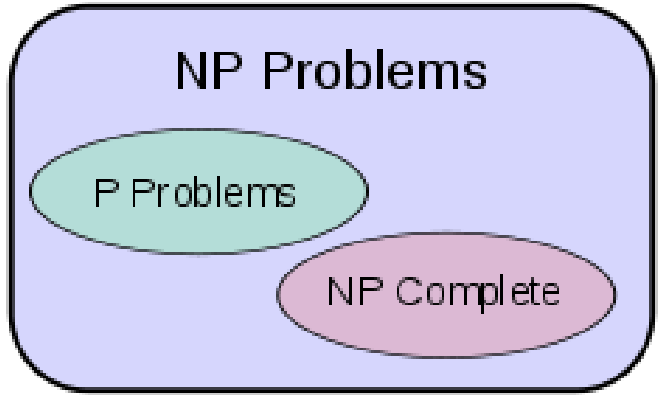
- Class NP: polynomial time checkable
  - L is in NP if there is a polynomial time algorithm M such that:

    x is in L iff ∃ y (polynomial length) s.t. M(x,y)=1



P versus NP

- One of the seven Millennium Prize Problems
- Most scientists believe P≠NP
  - Finding a proof is harder than checking a proof
- If P=NP
  - Most cryptosystems will be broken since there will be no one-way functions.





Classify NP problems

- P
  - Connectivity
  - Shortest path
  - Maximum flow
  - Maximum matching
  - Linear programming
  - Primality
  - Eulerian path
  - Edit distance
- in-between
  - Nash equilibrium
  - Graph isomorphism
  - Integer factoring
  - Parity game
- NP-Complete
  - Boolean satisfiability problem (Sat.)
  - Knapsack problem
  - Hamiltonian path
  - Travelling salesman problem
  - Subgraph isomorphism
  - Subset sum
  - Clique problem
  - Vertex cover
  - Independent set
  - Graph coloring



Problems harder than NP?

- Not easy to check a solution

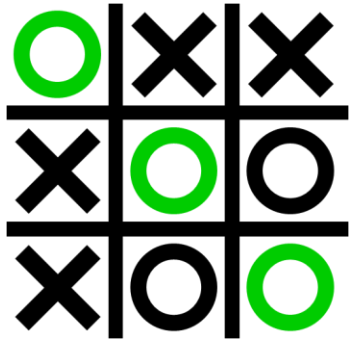




Problems harder than NP?

- PSPACE: solvable in polynomial space

$$\exists x_1 \, \forall x_2 \, \exists x_3 \, \forall x_4 : (x_1 \vee \neg x_3 \vee x_4) \wedge (\neg x_2 \vee x_3 \vee \neg x_4)$$

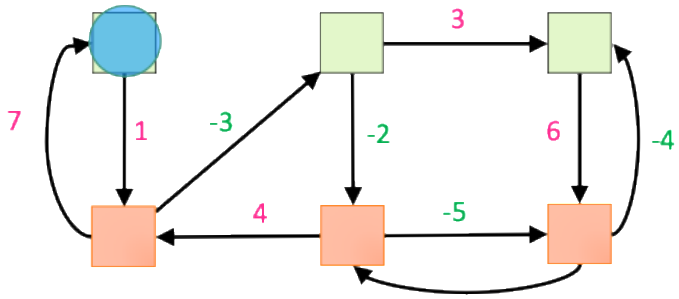


- EXPTIME: exponential time



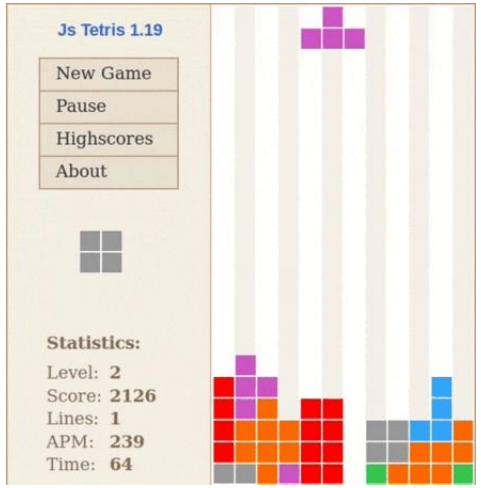
Complexity of Games *(generalized version)*

NP two-player game



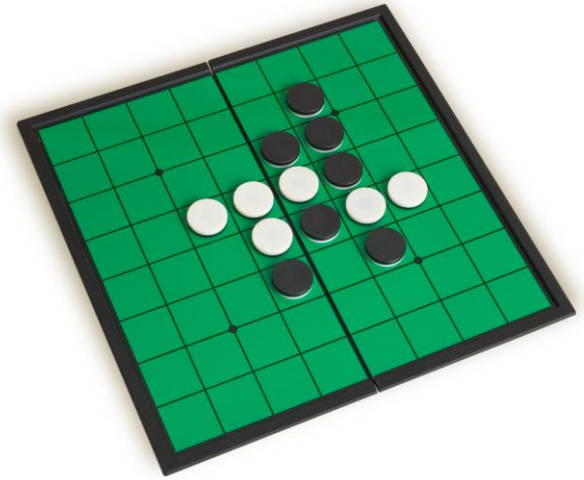
parity game

NP-complete one-player game



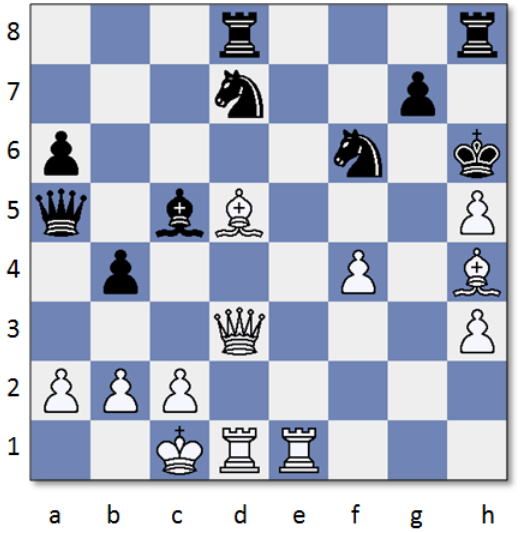
Tetris

PSPACE-complete



reversi

EXPTIME-complete

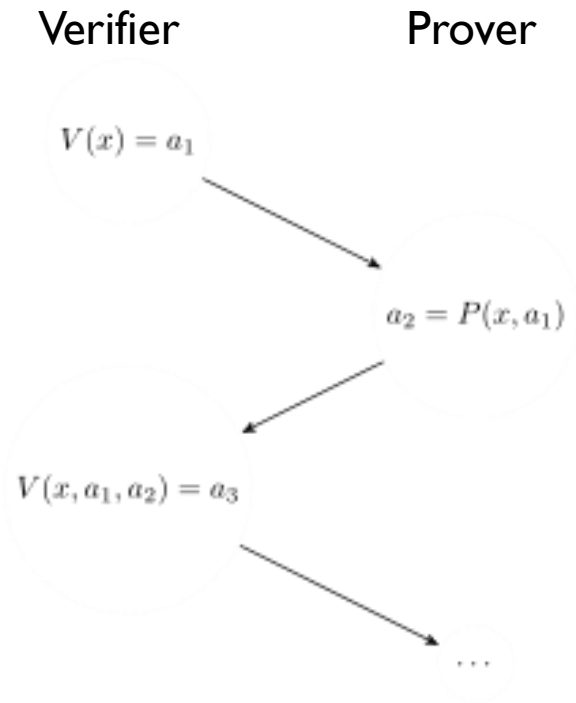


Chess



Interactive Proof

- How can you get more verifying ability than NP with limited computation?

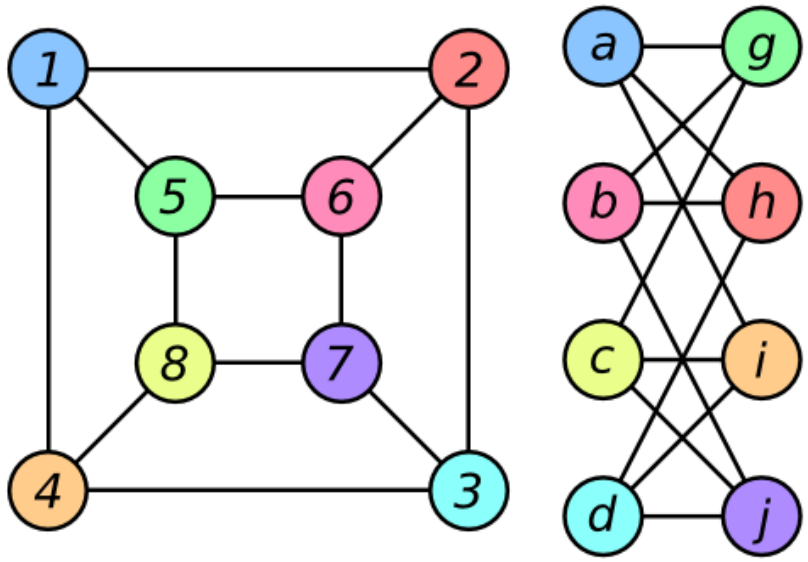




Interactive Proof

- We need randomness!

- Graph Isomorphism problem:





- Of course GI∈NP

- Graph nonisomorphism GNI∈co-NP
  - It's harder to verify!



- Suppose you want to distinguish G_1 and G_2

- V: pick $i \in \{1,2\}$ randomly. Randomly reorder G_i to get H. Send H to P.
- P: Identify which of G_j was used to product H. Send j to H.
- V:Check whether i=j.

- However, randomness is crucial!



- Computational theory:
  - Computable or not
  - *Weeks 1~5*

- Complexity theory:
  - How efficiently can the problem be computed?
  - Proving lower bound (for algorithms or data structures) is still very difficult
  - *Weeks 6~15*



Mathematical Logic

Formal Logical System

- Formal system:
  1. An alphabet of symbols
  2. A set of finite strings of these symbols, called *well-formed formula (wf.)*
  3. A set of well-formed formulas, called *axioms*
  4. A finite set of "*rules of deduction*"

- We can prove a "theorem" from axioms by the rules of deduction



Perfect Logical System

- Soundness
  - Every theorem in the system is “true”
  - (Computer don’t know what is “true”)
- Consistency
  - Doesn’t exist a statement A such that both A and (not A) are theorems
- Completeness
  - All true statements can be derived



Principia Mathematica

- Whitehead and Russell (1910–1913)
  - Three Volumes, 2000 pages
- Attempted to axiomatize mathematical reasoning
  - Define mathematical entities (like numbers) using logic
  - Derive mathematical "truths" by following mechanical rules of inference
  - Claimed to be *complete* and *consistent*
    - All true theorems could be derived
    - No falsehoods could be derived



Gödel's Solution

All consistent axiomatic formulations of number theory include *undecidable* propositions.

undecidable – cannot be proven either true or false inside the system.



Statement calculus

- In natural languages, there are simple sentences:
  - Napoleon is dead
  - John owes James two pounds
- Or compound sentences:
  - Napoleon is dead and the world is rejoicing
  - If all eggs are not square then all eggs are round
  - If the barometer falls then either it will rain or it will snow
- In two-valued logic, every simple sentence is either true or false!



Connectives

- For compound sentences:

Meaning	Connectives
not A	~A
A and B	A∧B
A or B	A∨B
if A then B	A→B
A if and only if B	A↔B



Connectives

- For compound sentences:

Meaning	Connectives
not A	~A
A and B	A∧B
A or B	A∨B
if A then B	A→B
A if and only if B	A↔B

  - Napoleon is dead and the world is rejoicing
    - A∧B
  - If all eggs are not square then all eggs are round
    - C→D
  - If the barometer falls then either it will rain or it will snow
    - E→(F∨G)



Truth function and truth table

p	~p
T	F
F	T

p	q	p∧q
T	T	T
T	F	F
F	T	F
F	F	F

p	q	p∨q
T	T	T
T	F	T
F	T	T
F	F	F



Truth function and truth table

- What about Conditional

p	q	p→q
T	T	T
T	F	?
F	T	?
F	F	?



Truth function and truth table

- What about Conditional

p	q	p→q
T	T	T
T	F	F
F	T	?
F	F	?



Truth function and truth table

- What about Conditional

p	q	p→q
T	T	T
T	F	F
F	T	T
F	F	T

- Truth of A infer truth of B
- Nothing is inferred from the falsity of A
  - For every integer n, if n>2 then $n^2>4$



Truth function and truth table

- Biconditional

p	q	p↔q
T	T	T
T	F	F
F	T	F
F	F	T



Tautology (重言式)

- A statement form is a tautology if it takes truth value T under each possible assignment of truth values to the statement variables which occur in it.
- A statement form is a contradiction if it takes truth value F under each possible assignment of truth values to the statement variables which occur in it.

- Example:
  - $(p \vee \sim p)$ is a tautology
  - $(q \wedge \sim q)$ is a contradiction



Adequate sets of connectives

- A set of connectives such that every truth function can be represented by a statement form containing only connectives from that set.

- Of course, {~, ∧, ∨} is an adequate set

- In fact, {~, ∧}, {~, ∨}, {~, →} are adequate set
  - $(\mathcal{A} \wedge \mathcal{B})$ is equivalent to $(\sim(\mathcal{A} \rightarrow \sim \mathcal{B}))$
  - $(\mathcal{A} \vee \mathcal{B})$ is equivalent to $(\sim\mathcal{A} \rightarrow \mathcal{B})$



Predicate calculus

- Examples:
  - All men are mortal
  - Socrates is a man
  - ∴ Socrates is mortal
- Universal quantifier: "for all x" (∀x)
- Existential quantifier: "there exists x" (∃x)

- How to write "All men are mortal"?
  - (∀x)(A(x)→M(x))
  - A: "is a man", M: "is mortal"
  - (∀x): for every object x in the universe



Relation of ∀ and ∃

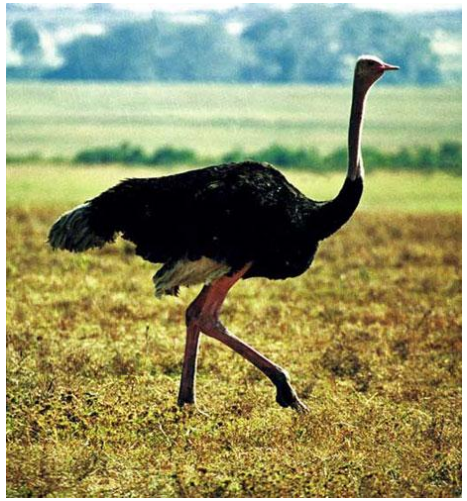
- "Not all birds can fly"
  - B(x):"is a bird"
  - F(x):"can fly"
  - Translates to: ~(∀x)(B(x)→F(x))





Relation of ∀ and ∃

- "Not all birds can fly" ⇔ "There is a bird that cannot fly"
  - B(x):"is a bird"
  - F(x):"can fly"
  - ~(∀x)(B(x)→F(x)) ⇔ (∃x)(B(x)∧~F(x))







Relation of ∀ and ∃

- "Not all birds can fly" ⇔ "There is a bird that cannot fly"
  - B(x):"is a bird"
  - F(x):"can fly"
  - ~(∀x)(B(x)→F(x)) ⇔ (∃x)(B(x)∧~F(x))

- We know:
  - ~(∀x)(B(x)→F(x))
  - ⇔ ~(∀x)(~B(x)∨F(x))
  - ⇔ ~(∀x)~ (B(x)∧~F(x))



Relation of ∀ and ∃

- "Not all birds can fly" ⇔ "There is a bird that cannot fly"
  - B(x):"is a bird"
  - F(x):"can fly"
  - ~(∀x)(B(x)→F(x)) ⇔ (∃x)(B(x)∧~F(x))

- In fact, $(\exists x)\mathcal{A}$ is an abbreviation for $\sim(\forall x)\sim\mathcal{A}$
  - "It's not the case that all x's do not have property P"
  - is equivalent to "there exists an x which has property P"



First-order languages

- The alphabet of symbols:

Names	Symbols
variables	$x_1, x_2, \ldots$
individual constants	$a_1, a_2, \ldots$
predicate letters	A_i^n,
function letters	f_i^n,
punctuation symbols	(,) and ,
connectives	~ and $\rightarrow$
the quantifier	$\forall$



First-order languages

- The alphabet of symbols:

Names	Symbols
variables	$x_1, x_2, \ldots$
individual constants	$a_1, a_2, \ldots$
predicate letters	A_i^n,
function letters	f_i^n,
punctuation symbols	(,) and ,
connectives	~ and $\rightarrow$
the quantifier	$\forall$

Predicate is like a relation R

- Examples
  - $(\forall x_1)(\forall x_2)A_1^2(f_1^2(x_1,x_2), f_1^2(x_2,x_1))$
  - If A_1^2 stands for =, f_1^2 stands for +, it is $(\forall x_1)(\forall x_2)(x_1+x_2=x_2+x_1)$



First-order language $\mathcal{L}$

- A term:
  - variables and constants are terms
  - if $t_1, t_2, \ldots, t_n$ are terms, then $f_i^n(t_1, t_2, \ldots, t_n)$ is a term
- Atomic formula:
  - if $t_1, t_2, \ldots, t_n$ are terms, then $A_i^n(t_1, t_2, \ldots, t_n)$ is an atomic formula
- A well-formed formula of $\mathcal{L}$
  - every atomic formula of $\mathcal{L}$
  - if $\mathcal{A}$ and $\mathcal{B}$ are wfs. of $\mathcal{L}$, so are $(\sim \mathcal{A})$, $(\mathcal{A} \rightarrow \mathcal{B})$ and $(\forall x_i)\, \mathcal{A}$, where x_i is any variable



Scope

- In $(\forall x_i)\,\mathcal{A}$, we say that $\mathcal{A}$ is the scope of the quantifier
- An occurrence of x_i in a wf. is said to be **bound** if it occurs within the scope of a $(\forall x_i)$ in the wf. or if it is the x_i in a $(\forall x_i)$
- Otherwise it is **free**



Scope

- In $(\forall x_i)\,\mathcal{A}$, we say that $\mathcal{A}$ is the scope of the quantifier
- An occurrence of x_i in a wf. is said to be **bound** if it occurs within the scope of a $(\forall x_i)$ in the wf. or if it is the x_i in a $(\forall x_i)$
- Otherwise it is **free**
- Example: $(\forall x_1)(A_1^2(x_1, x_2) \rightarrow (\forall x_2)\, A_1^1(x_2))$



Scope

- In $(\forall x_i)\,\mathcal{A}$, we say that $\mathcal{A}$ is the scope of the quantifier
- An occurrence of x_i in a wf. is said to be **bound** if it occurs within the scope of a $(\forall x_i)$ in the wf. or if it is the x_i in a $(\forall x_i)$
- Otherwise it is **free**
- Example: $(\forall x_1)(A_1^2(x_1, x_2) \rightarrow (\forall x_2)\, A_1^1(x_2))$



Substitution

- In mathematics, from $(\forall x_1)(\exists x_2)(x_2=x_1\cdot(x_1+1))$, which one is not true?
  A. $(\exists x_2)(x_2=0\cdot(0+1))$
  B. $(\exists x_2)(x_2=x_3\cdot(x_3+1))$
  C. $(\exists x_2)(x_2=f(x_1,x_3)\cdot(f(x_1,x_3)+1))$
  D. $(\exists x_2)(x_2=f(x_1,x_2)\cdot(f(x_1,x_2)+1))$



Substitution

- In mathematics, from $(\forall x_1)(\exists x_2)(x_2=x_1\cdot(x_1+1))$, which one is not true?
  - A. $(\exists x_2)(x_2=0\cdot(0+1))$
  - B. $(\exists x_2)(x_2=x_3\cdot(x_3+1))$
  - C. $(\exists x_2)(x_2=f(x_1,x_3)\cdot(f(x_1,x_3)+1))$
  - D. $(\exists x_2)(x_2=f(x_1,x_2)\cdot(f(x_1,x_2)+1))$



- we should be careful when using $(\forall x_1)\mathcal{A}(x_1)\rightarrow\mathcal{A}(t)$ for a term t



Interpretations

- An interpretation I of $\mathcal{L}$ is a non-empty set D_I (the domain of I) together with a collection of distinguished elements, a collection of functions on D_I, and a collection of relations on D_I.

Names	Symbols
variables	$x_1, x_2, \ldots$
individual constants	$a_1, a_2, \ldots$
predicate letters	A_i^n,
function letters	f_i^n,
punctuation symbols	(,) and ,
connectives	~ and $\rightarrow$
the quantifier	$\forall$

Not specified!



Interpretations

- An interpretation I of $\mathcal{L}$ is a non-empty set D_I (the domain of I) together with a collection of distinguished elements, a collection of functions on D_I, and a collection of relations on D_I.

- Example:
  - $D_I=\{0,1,2,\ldots\}$
  - a_1 is 0
  - A_1^2 stands for =, f_1^2 stands for +,
  - Then $(\forall x_I)A_I^2(f_I^2(x_I,a_I),x_I)$ is **true** in I



Satisfaction

- A valuation in I is a function v from the set of variables of $\mathcal{L}$ to the set D_I.
  - We call a valuation in I "satisfies" $\mathcal{A}$.

- Example:
  - $v(x_1)=1, v(x_2)=1, v(x_3)=2$ satisfies $x_1+x_2=x_3$
  - $v(x_1)=2, v(x_2)=3, v(x_3)=1$ doesn't satisfy $x_1+x_2=x_3$

- What about quantifiers?
  - We can ignore the valuation of x_1 when it appears in $(\forall x_1)\mathcal{A}$
  - $v(x_1)=2, v(x_2)=3$ satisfies $(\forall x_1)(\forall x_2)(x_1+x_2=x_2+x_1)$



Satisfaction

- A valuation in I is a function v from the set of variables of $\mathcal{L}$ to the set D_I
  - We call a valuation in I "satisfies" $\mathcal{A}$

- True:
  - $\mathcal{A}$ is true in an interpretation in I if every valuation in I satisfies $\mathcal{A}$
  - $\mathcal{A}$ is false if there is no valuation in I which satisfies $\mathcal{A}$

- Logically valid
  - $\mathcal{A}$ is logically valid if $\mathcal{A}$ is true in every interpretation of L
  - $\mathcal{A}$ is contradictory if it is false in every interpretation



- True:
  - a wf. $\mathcal{A}$ is true in an interpretation in I if every valuation in I satisfies $\mathcal{A}$
  - $\mathcal{A}$ is false if there is no valuation in I which satisfies $\mathcal{A}$

- Logically valid
  - a wf. $\mathcal{A}$ is logically valid if $\mathcal{A}$ is true in every interpretation of L
  - $\mathcal{A}$ is contradictory if it is false in every interpretation

Examples of logically valid wfs.:

$(\forall x_1)A_1^1(x_1) \rightarrow (\forall x_2)A_1^1(x_2)$

$(\forall x_1)A_1^1(x_1) \rightarrow A_1^1(x_1)$

$(\forall x_1)A_1^1(x_1) \rightarrow A_1^1(x_2)$

$(\forall x_1)(A_1^1(x_2) \rightarrow A_1^2(x_2, x_1)) \rightarrow (A_1^1(x_2) \rightarrow (\forall x_1)A_1^2(x_2, x_1))$



Closed form

- a wf. $\mathcal{A}$ of $\mathcal{L}$ is closed if no variable occurs free in $\mathcal{A}$



Closed form

- a wf. $\mathcal{A}$ of $\mathcal{L}$ is closed if no variable occurs free in $\mathcal{A}$

  - $(\forall x_1)(\forall x_2)A_1^2(f_1^2(x_1,x_2), f_1^2(x_2,x_1))$ -- closed
  - $(\forall x_1)A_1^1(x_1) \rightarrow A_1^1(x_1)$ -- NOT closed

- If $\mathcal{A}$ is a closed wf. of $\mathcal{L}$ and I is an interpretation of $\mathcal{L}$, then either $\mathcal{A}$ is true or false in I



Formal predicate calculus

- Formal deductive system
  - We can construct an argument by proof
  - not evaluating truth table

- Just like in Euclidean geometry:

Statements	Reasons
1. $\overline{AC} \cong \overline{EC}$	1. Given
2. $\overline{BC} \cong \overline{DC}$	2. Given
3. $\angle ACB \cong \angle ECD$	3. Vertical Angles Theorem
4. $\triangle ABC \cong \triangle EDC$	4. SAS Postulate



Formal predicate calculus

- Formal deductive system
  - We can construct an argument by proof
  - not evaluating truth table

- Formal system:
  1. An alphabet of symbols
  2. A set of finite strings of these symbols, called *well-formed formula (wf.)*
  3. A set of well-formed formulas, called *axioms*
  4. A finite set of "*rules of deduction*"



Formal predicate calculus

Formal system $K_{\mathcal{L}}$

- Axioms
  - *(K1)* $(\mathcal{A} \rightarrow (\mathcal{B} \rightarrow \mathcal{A}))$
  - *(K2)* $((\mathcal{A} \rightarrow (\mathcal{B} \rightarrow \mathcal{C})) \rightarrow ((\mathcal{A} \rightarrow \mathcal{B}) \rightarrow (\mathcal{A} \rightarrow \mathcal{C})))$
  - *(K3)* $(((\sim \mathcal{A}) \rightarrow (\sim \mathcal{B})) \rightarrow (\mathcal{B} \rightarrow \mathcal{A}))$
  - *(K4)* $((\forall x_i)\, \mathcal{A} \rightarrow \mathcal{A})$, if x_i does not occur free in $\mathcal{A}$
  - *(K5)* $((\forall x_i)\, \mathcal{A}(x_i) \rightarrow \mathcal{A}(t))$, if t is a term in $\mathcal{L}$ which is free for x_i in $\mathcal{A}(x_i)$
  - *(K6)* $(\forall x_i)(\mathcal{A} \rightarrow \mathcal{B}) \rightarrow (\mathcal{A} \rightarrow (\forall x_i)\mathcal{B})$, if $\mathcal{A}$ contains no free occurrence of x_i
  - *each axiom scheme has infinite many "instances"*
- Rules
  1. Modus ponens: from $\mathcal{A}$ and $(\mathcal{A} \rightarrow \mathcal{B})$ deduce $\mathcal{B}$
  2. Generalisation: from $\mathcal{A}$ deduce $(\forall x_i)\, \mathcal{A}$, where x_i is any variable



Proof and deduction

- A proof in $K_{\mathcal{L}}$ is a sequence of wfs. $\mathcal{A}_1, \ldots \mathcal{A}_n$, such that each $\mathcal{A}_i$ is either:
  - an axiom of $\mathcal{L}$
  - follows from $\mathcal{A}_j$ and $\mathcal{A}_k$ *(j,k<i)* as a consequence by MP or Generalisation
- It is a proof of $\mathcal{A}_n$ in $\mathcal{L}$, $\mathcal{A}_n$ is a theorem
- denoted $\vdash_K \mathcal{A}_n$

Exercise:

$\vdash_L (\mathcal{A} \rightarrow \mathcal{A})$ (only L1 and L2)



Proof and deduction

- A deduction in $K_{\mathcal{L}}$ from a set of wfs. Γ is a sequence of wfs. $\mathcal{A}_1, \ldots \mathcal{A}_n$, such that each $\mathcal{A}_i$ is either:
  - an axiom of $\mathcal{L}$
  - a member of Γ
  - follows from $\mathcal{A}_j$ and $\mathcal{A}_k$ *(j,k<i)* as a consequence by MP or Generalisation
- denoted $\Gamma \vdash_K \mathcal{A}_n$



Example

- If x_i occurs free in $\mathcal{A}(x_i)$ and x_j is a variable which does not occur in $\mathcal{A}(x_i)$, then
  - $\vdash_K (\forall x_i)\mathcal{A}\ (x_i) \rightarrow (\forall x_j)\mathcal{A}\ (x_j)$



Example

- If x_i occurs free in $\mathcal{A}(x_i)$ and x_j is a variable which does not occur in $\mathcal{A}(x_i)$, then
  - $\vdash_K (\forall x_i)\mathcal{A}\ (x_i) \rightarrow (\forall x_j)\mathcal{A}\ (x_j)$

$(\forall x_i)\mathcal{A}\ (x_i) \rightarrow \mathcal{A}\ (x_j)$	(K5)
$(\forall x_j)((\forall x_i)\mathcal{A}\ (x_i) \rightarrow \mathcal{A}\ (x_j))$	(Generalisation rule)
$(\forall x_j)((\forall x_i)\mathcal{A}\ (x_i) \rightarrow \mathcal{A}\ (x_j)) \rightarrow ((\forall x_i)\mathcal{A}\ (x_i) \rightarrow (\forall x_j)\mathcal{A}\ (x_j))$	(K6)
$(\forall x_i)\mathcal{A}\ (x_i) \rightarrow (\forall x_j)\mathcal{A}\ (x_j)$	(MP)



“Hypothetical Syllogism” (HS)

- $\{(\mathcal{A} \rightarrow \mathcal{B}), (\mathcal{B} \rightarrow \mathcal{C})\} \vdash_L (\mathcal{A} \rightarrow \mathcal{C})$

- proof by the deduction theorem

- Exercise:

1. $\vdash_L (\sim \mathcal{B} \rightarrow (\mathcal{B} \rightarrow \mathcal{A}))$
2. $\vdash_L ((\sim \mathcal{A} \rightarrow \mathcal{A}) \rightarrow \mathcal{A})$ (difficult)



Adequacy theorem for $K_{\mathcal{L}}$

- Soundness
  - Every theorem of $K_{\mathcal{L}}$ is logically valid
- Consistency
  - No wf. $\mathcal{A}$ of $K_{\mathcal{L}}$ such that both $\mathcal{A}$ and $(\sim \mathcal{A})$ are theorems
- Adequacy
  - If $\mathcal{A}$ is a wf. and $\mathcal{A}$ is logically valid, then $\mathcal{A}$ is a theorem in $K_{\mathcal{L}}$

Logically valid ⇔ Theorem in $K_{\mathcal{L}}$



Mathematical systems

- We add "proper axioms" to $K_{\mathcal{L}}$, besides logical axioms
- A consistent extension of $K_{\mathcal{L}}$



- Axioms for equality "="
  - (E7) $x_l = x_l$
  - (E8) $(t_k = u) \rightarrow (f_i^n(t_l,\ldots,t_k,\ldots,t_n)=f_i^n(t_l,\ldots,u,\ldots t_n))$
  - (E9) $(t_k = u) \rightarrow (A_i^n(t_l,\ldots,t_k,\ldots,t_n) \rightarrow A_i^n(t_l,\ldots,u,\ldots t_n))$



First order arithmetic $\mathcal{N}$

- (N1) $(\forall x_1) \sim(x_1'=0)$
- (N2) $(\forall x_1)(\forall x_2)\ (x_1'=x_2' \rightarrow x_1=x_2)$
- (N3) $(\forall x_1)\ (x_1+0=x_1)$
- (N4) $(\forall x_1)(\forall x_2)((x_1+ x_2')=(x_1+x_2)')$
- (N5) $(\forall x_1)\ (x_1 \times 0=0)$
- (N6) $(\forall x_1)(\forall x_2)((x_1 \times x_2')=(x_1 \times x_2)+x_1)$
- (N7) $\mathcal{A}(0) \rightarrow((\forall x_1)(\mathcal{A}(x_1) \rightarrow \mathcal{A}(x_1'))\rightarrow (\forall x_1)\mathcal{A}(x_1))$, where x_1 occurs free in $\mathcal{A}(x_1)$

- Peano's Postulates.



$\mathcal{N}$: $K_{\mathcal{L}}$ with (E7)-(E9), (N1)-(N7)

- Prove: (0” × 0”=0””)

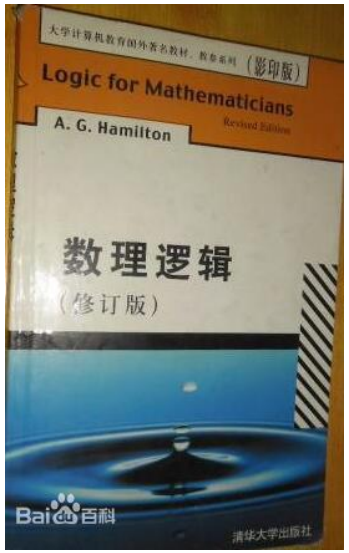
(E7) $x_1 = x_1$
(E8) $t_k = u \rightarrow f_i^n(t_1,\ldots,t_k,\ldots,t_n) = f_i^n(t_1,\ldots,u,\ldots t_n)$
(E9) $t_k = u \rightarrow (A_i^n(t_1,\ldots,t_k,\ldots,t_n) \rightarrow A_i^n(t_1,\ldots,u,\ldots t_n))$

(N1) $(\forall x_1) \sim(x_1' = 0)$
(N2) $(\forall x_1)(\forall x_2)\ (x_1' = x_2' \rightarrow x_1 = x_2)$
(N3) $(\forall x_1)\ (x_1 + 0 = x_1)$
(N4) $(\forall x_1)(\forall x_2)((x_1 + x_2') = (x_1 + x_2)')$
(N5) $(\forall x_1)\ (x_1 \times 0 = 0)$
(N6) $(\forall x_1)(\forall x_2)((x_1 \times x_2') = (x_1 \times x_2) + x_1)$
(N7) $\mathcal{A}(0) \rightarrow ((\forall x_1)(\mathcal{A}(x_1) \rightarrow \mathcal{A}(x_1')) \rightarrow (\forall x_1)\mathcal{A}(x_1))$, where x_1 occurs free in $\mathcal{A}(x_1)$



Note

- Formally only 0 is a symbol in $\mathcal{N}$, 1,2,3,… are not
- 1,2,3, … , n, are represented by 0', 0'', 0''',… , $0^{(n)}$, …
- Here for convenience we just use n





Gödel incompleteness theorem

Kurt Gödel

Kurt Gödel, born April 28, 1906, was an Austrian - American logician , mathematician and philosopher . One of the most significant logicians of all time, Gödel made an immense impact upon scientific and philosophical thinking in the 20th century.

Gödel is best known for his two incompleteness theorems, published in 1931 when he was 25 years of age, one year after finishing his doctorate at the University of Vienna.



Historical Perspective

- In the nineteenth and early twentieth centuries, one of the big mathematical goals was to reduce all of number theory to a formal axiomatic system.
- Russell and Whitehead's *Principia Mathematica* was the most famous attempt
- Gödel's theorem dashed this hope completely
- Gödel showed that for any formal axiomatic system *capable of expressing elementary arithmetic*, *there is always a statement about natural numbers which is true, but which cannot be proven in the system*



- Hilbert's 2nd problem:
  - Prove that the axioms of arithmetic are consistent

- More generally, is the first-order arithmetic system $\mathcal{N}$ complete?

  - consistent: there is no $\mathcal{A}$ such that $\vdash_{\mathcal{N}} \mathcal{A}$ and $\vdash_{\mathcal{N}} \sim\mathcal{A}$
  - complete: for all closed wf. $\mathcal{U}$, either $\vdash_{\mathcal{N}} \mathcal{U}$ or $\vdash_{\mathcal{N}} \sim\mathcal{U}$



- Closed wf.: mathematical statement
  - $(\forall x_1)((x_1 \times x_1') = (x_1 \times x_1) + x_1)$
  - either true or false

- wf. with free variables: relation between free variables
  - $(\exists x_3)(x_1 + x_3 = x_2)$
  - (natural numbers) "$x_1 \leq x_2$"



General recursive function

- Functions on natural numbers $\mathbb{N} \to \mathbb{N}$
- Three basic functions
- Three rules:
  - Composition. Given the functions $g(n_1,\ldots,n_j)$ and $h_i(n_1,\ldots,n_k)$ for $1 \le i \le j$, then we have $f(n_1,\ldots,n_k)=g(h_1(n_1,\ldots,n_k),\ldots, h_j(n_1,\ldots,n_k))$.
  - Recursion. Given $g(n_1,\ldots,n_k)$ and $h(n_1,\ldots,n_{k+2})$, we have

    $f(n_1,\ldots,n_k,0)= g(n_1,\ldots,n_k)$

    $f(n_1,\ldots,n_k,n+1)= h(n_1,\ldots,n_k,n, f(n_1,\ldots,n_k,n))$
  - Minimisation. Given $g(m,n_1,\ldots,n_k)$, we define

    $f(n_1,\ldots,n_k)=p$ iff $g(p,n_1,\ldots,n_k)=0$, and $g(i,n_1,\ldots,n_k)>0$ for $0 \le i < p$

- recursive $\Leftrightarrow$ decidable by Turing machine



Properties of $\mathcal{N}$

- Remind $\vdash_{\mathcal{N}} (\sim \mathcal{B} \rightarrow (\mathcal{B} \rightarrow \mathcal{A}))$
- Extension $\mathcal{N}^+$ of $\mathcal{N}$: we add more axioms to $\mathcal{N}$
  - Theorems in $\mathcal{N}$ remains theorems in $\mathcal{N}^+$
- If $\mathcal{N}^+$ is inconsistent
  - that is, $\vdash_{\mathcal{N}^+} \mathcal{B}$ and $\vdash_{\mathcal{N}^+} \sim \mathcal{B}$
  - then $\vdash_{\mathcal{N}^+} \mathcal{A}$ for any wf. $\mathcal{A}$
  - inconsistent system can prove anything, so the system is meaningless



Properties of $\mathcal{N}$

- There are infinite many axioms in $\mathcal{N}$

(K1) $(\mathcal{A} \rightarrow (\mathcal{B} \rightarrow \mathcal{A}))$
(K2) $((\mathcal{A} \rightarrow (\mathcal{B} \rightarrow \mathcal{C})) \rightarrow ((\mathcal{A} \rightarrow \mathcal{B}) \rightarrow (\mathcal{A} \rightarrow \mathcal{C})))$
(K3) $(((\sim \mathcal{A}) \rightarrow (\sim \mathcal{B})) \rightarrow (\mathcal{B} \rightarrow \mathcal{A}))$
(K4) $((\forall x_i)\, \mathcal{A} \rightarrow \mathcal{A})$, if x_i does not occur free in $\mathcal{A}$
(K5) $((\forall x_i)\, \mathcal{A}(x_i) \rightarrow \mathcal{A}(t))$, if t is a term in $\mathcal{L}$ which is free for x_i in $\mathcal{A}(x_i)$
(K6) $(\forall x_i)(\mathcal{A} \rightarrow \mathcal{B}) \rightarrow (\mathcal{A} \rightarrow (\forall x_i)\mathcal{B})$, if $\mathcal{A}$ contains no free occurrence of x_i

- But the set of axioms is recursive
  - we can use a TM to list all of them
  - Given a formula, checking whether it is an axiom is recursive (decidable)



Properties of $\mathcal{N}$

- Proofs are "convincing": a TM can verify a candidate proof of a theorem

- $\{(\mathcal{U},\mathcal{P}) \mid \mathcal{P}$ is a proof of $\mathcal{U}$ in $\mathcal{N}\}$ is recursive



Properties of $\mathcal{N}$

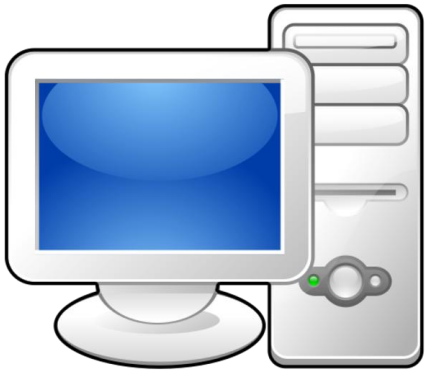
- Mathematical statements about computation can be (computably) described as a statement of $\mathcal{N}$
- Given Turing machine M and input w, there is a computable wf. $\mathcal{S}$(M(w)) in $\mathcal{N}$ such that
  $\vdash_{\mathcal{N}} \mathcal{S}$(M(w)) if and only if M accepts w
- Then if M rejects w and M always halts, $\vdash_{\mathcal{N}} \sim\mathcal{S}$(M(w))



Gödel numbering

- Encode all wfs. to numbers, including a "proof"
  - We use $\langle \mathcal{A} \rangle$ to denote the Gödel number of wfs. $\mathcal{A}$

- In modern computers, everything is encoded by a number!







Gödel numbering

- Given Turing machine M and input w, there is a computable wf. $\mathcal{S}(\mathrm{M}(\mathrm{w}))$ in $\mathcal{N}$ such that

  $\vdash_{\mathcal{N}} \mathcal{S}(\mathrm{M}(\mathrm{w}))$ if and only if M accepts w

- Pf($\mathcal{U}$,$\mathcal{P}$) is recursive: $\mathcal{P}$ is a proof in $\mathcal{N}$ of $\mathcal{U}$

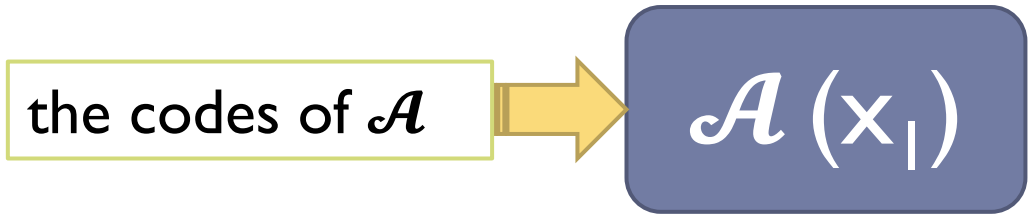
- So there is a wf. $\mathcal{F}(x_1,x_2)$ such that:
  - If Pf($\mathcal{U}$,$\mathcal{P}$) holds then $\vdash_{\mathcal{N}} \mathcal{F}(\langle\mathcal{U}\rangle,\langle\mathcal{P}\rangle)$
  - If Pf($\mathcal{U}$,$\mathcal{P}$) does not hold then $\vdash_{\mathcal{N}} \sim\mathcal{F}(\langle\mathcal{U}\rangle,\langle\mathcal{P}\rangle)$



Gödel Incompleteness Theorem

- $W(\mathcal{A},\mathcal{P})$ is recursive
  - $\mathcal{A}(x_1)$ is a wf. in which only x_1 occurs free
  - $\langle\mathcal{A}\rangle$ is the Gödel number of the wf. $\mathcal{A}(x_1)$
  - $\mathcal{P}$ is a proof in $\mathcal{N}$ of $\mathcal{A}(\langle\mathcal{A}\rangle)$

- $\mathcal{A}(\langle\mathcal{A}\rangle)$ is closed form





Gödel Incompleteness Theorem

- $W(\mathcal{A},\mathcal{P})$ is recursive
  - $\mathcal{A}(x_1)$ is a wf. in which only x_1 occurs free
  - $\langle\mathcal{A}\rangle$ is the Gödel number of the wf. $\mathcal{A}(x_1)$
  - $\mathcal{P}$ is a proof in $\mathcal{N}$ of $\mathcal{A}(\langle\mathcal{A}\rangle)$

- So there is a wf. $\mathcal{W}(x_1,x_2)$ such that:
  - If $W(\mathcal{A},\mathcal{P})$ holds then $\vdash_{\mathcal{N}} \mathcal{W}(\langle\mathcal{A}\rangle,\langle\mathcal{P}\rangle)$
  - If $W(\mathcal{A},\mathcal{P})$ does not hold then $\vdash_{\mathcal{N}} \sim\mathcal{W}(\langle\mathcal{A}\rangle,\langle\mathcal{P}\rangle)$



Gödel Incompleteness Theorem

- Idea:
  - "This sentence cannot be proved"
  - If it can be proved, then it is wrong, so we can prove something not correct.
  - If it cannot be proved, it is correct, so there is something correct that cannot be proved.

- Barber's paradox:
  - The barber is the "one who shaves all those, and those only, who do not shave themselves." The question is, does the barber shave himself?



Proof

- $W(\mathcal{A},\mathcal{P})$ is recursive
  - $\mathcal{A}(x_1)$ is a wf. in which only x_1 occurs free
  - $\langle\mathcal{A}\rangle$ is the Gödel number of the wf. $\mathcal{A}(x_1)$
  - $\mathcal{P}$ is a proof in $\mathcal{N}$ of $\mathcal{A}(\langle\mathcal{A}\rangle)$

- Let $\mathcal{U}(x_1)$ be wf. $(\forall x_2)\sim\mathcal{W}(x_1,x_2)$
  - Only x_1 occurs free in $(\forall x_2)\sim\mathcal{W}(x_1,x_2)$

- Consider $\mathcal{U}(\langle\mathcal{U}\rangle)$:

If $\mathcal{P}$ is a proof of $\mathcal{U}(\langle\mathcal{U}\rangle)$, which is $(\forall x_2)\sim\mathcal{W}(\langle\mathcal{U}\rangle,x_2)$
then $W(\mathcal{U},\mathcal{P})$ holds, and $\vdash_{\mathcal{N}} \mathcal{W}(\langle\mathcal{U}\rangle,\langle\mathcal{P}\rangle)$
but since there is a proof of $\mathcal{U}(\langle\mathcal{U}\rangle)$, that is, $\vdash_{\mathcal{N}} (\forall x_2)\sim\mathcal{W}(\langle\mathcal{U}\rangle,x_2)$
so $\vdash_{\mathcal{N}} \sim\mathcal{W}(\langle\mathcal{U}\rangle,\langle\mathcal{P}\rangle)$, contradicting the consistency of $\mathcal{N}$



Proof

- $W(\mathcal{A},\mathcal{P})$ is recursive
  - $\mathcal{A}(x_1)$ is a wf. in which only x_1 occurs free
  - $\langle\mathcal{A}\rangle$ is the Gödel number of the wf. $\mathcal{A}(x_1)$
  - $\mathcal{P}$ is a proof in $\mathcal{N}$ of $\mathcal{A}(\langle\mathcal{A}\rangle)$

- Let $\mathcal{U}(x_1)$ be wf. $(\forall x_2)\sim\mathcal{W}(x_1,x_2)$
  - Only x_1 occurs free in $(\forall x_2)\sim\mathcal{W}(x_1,x_2)$

- Consider $\mathcal{U}(\langle\mathcal{U}\rangle)$:

So there is no proof of $\mathcal{U}(\langle\mathcal{U}\rangle)$, which is $(\forall x_2)\sim\mathcal{W}(\langle\mathcal{U}\rangle,x_2)$
then for all $\mathcal{P}$, $W(\mathcal{U},\mathcal{P})$ does not hold and $\vdash_{\mathcal{N}} \sim\mathcal{W}(\langle\mathcal{U}\rangle,\langle\mathcal{P}\rangle)$
Assume $\mathcal{N}$ is ω-consistent



ω-consistency

- first order system S is consistent iff
  - no wf. $\mathcal{A}$ s.t. both $\mathcal{A}$ and $(\sim\mathcal{A})$ are theorems in S

- first order system S is ω-consistent iff no wf. $\mathcal{A}(x_1)$, in which x_1 occurs free, s.t. $\sim(\forall x_1)\mathcal{A}(x_1)$ and all of $\mathcal{A}(n)$ *(for every n)* are theorems in S
  - Slightly stronger



Proof

- $W(\mathcal{A},\mathcal{P})$ is recursive
  - $\mathcal{A}(x_1)$ is a wf. in which only x_1 occurs free
  - $\langle\mathcal{A}\rangle$ is the Gödel number of the wf. $\mathcal{A}(x_1)$
  - $\mathcal{P}$ is a proof in $\mathcal{N}$ of $\mathcal{A}(\langle\mathcal{A}\rangle)$

- Let $\mathcal{U}(x_1)$ be wf. $(\forall x_2)\sim\mathcal{W}(x_1,x_2)$
  - Only x_1 occurs free in $(\forall x_2)\sim\mathcal{W}(x_1,x_2)$

- Consider $\mathcal{U}(\langle\mathcal{U}\rangle)$:

So there is no proof of $\mathcal{U}(\langle\mathcal{U}\rangle)$, which is $(\forall x_2)\sim\mathcal{W}(\langle\mathcal{U}\rangle,x_2)$
then for all $\mathcal{P}$, $W(\mathcal{U},\mathcal{P})$ does not hold and $\vdash_{\mathcal{N}} \sim\mathcal{W}(\langle\mathcal{U}\rangle,\langle\mathcal{P}\rangle)$
By ω-consistency, $\sim(\forall x_2)\sim\mathcal{W}(\langle\mathcal{U}\rangle,x_2)$ is not a theorem in $\mathcal{N}$
Then both $\mathcal{U}(\langle\mathcal{U}\rangle)$ and $\sim\mathcal{U}(\langle\mathcal{U}\rangle)$ are not theorems in $\mathcal{N}$



Gödel Incompleteness Theorem

- CONCLUSION: $(\mathcal{T}=\mathcal{U}(\langle\mathcal{U}\rangle))$
  - If $\mathcal{T}$ is a theorem in $\mathcal{N}$, then $\mathcal{N}$ is not ω-consistent
  - Assume $\mathcal{N}$ is ω-consistent, then both $\mathcal{T}$ and $\sim\mathcal{T}$ are not theorems of $\mathcal{N}$
  - Hence, any ω-consistent $\mathcal{N}$ is not complete
  - (Gödel first incompleteness theorem)



- Can we extend $\mathcal{N}$ to be complete by adding new axioms?



- Can we extend $\mathcal{N}$ to be complete by adding new axioms?

- As long as checking a proof is recursive, we can construct such a non-provable $\mathcal{T}$ and $\sim\mathcal{T}$
  - requiring that the set of axioms is recursive

Note that our axiom schemes are like: $(\mathcal{A}\rightarrow(\mathcal{B}\rightarrow\mathcal{A}))$
It contains infinite many instances since $\mathcal{A}$ and $\mathcal{B}$ can be any wfs.
But all of them can be generated by a Turing machine



- Let $\mathcal{N}^+$ be any extension of $\mathcal{N}$ such that the Gödel numbers of axioms of $\mathcal{N}^+$ is a recursive set, then $\mathcal{N}^+$ cannot be both complete and consistent
  - recursive set: meaning that we can generate all axioms by a Turing machine

- Can we extend $\mathcal{N}$ to a complete and consistent system?
  - Yes, by the method in the proof of Gödel completeness theorem (for first-order logic)
  - but the axiom set cannot be recursive

Knowledge is infinite: there is no TM that can generate all truth



Limitations on Mathematics 😓

- Gödel First Incompleteness Theorem:
  - Any effectively generated theory capable of expressing elementary arithmetic cannot be both consistent and complete
- Gödel Second Incompleteness Theorem:
  - For any formal effectively generated theory T including basic arithmetical truths and also certain truths about formal provability, if T includes a statement of its own consistency then T is inconsistent
- *Entscheidungsproblem:*
  - The problem of checking whether a formula has a proof in first-order logic $K_{\mathcal{L}}$,
  - Alonzo Church and Alan Turing in 1936 shows that it is undecidable



Thank you!